

LAPORAN PRAKTIKUM PENGEMBANGAN APLIKASI MOBILE



Dosen Pengampu :

Muhammad Aminul Akbar, S.Kom., M.T.

Disusun Oleh :

Nama : Nada Halwa Yuanita

NIM : 245150407111021

PROGRAM STUDI SISTEM INFORMASI

JURUSAN SISTEM INFORMASI

FAKULTAS ILMU KOMPUTER

UNIVERSITAS BRAWIJAYA

MALANG

2026

MODUL 4

STATE HOISTING, PERUBAHAN KONFIGURASI, DAN LAUNCHED EFFECT PADA JETPACK COMPOSE

A. Percobaan 1 State Hoisting

Pada percobaan ini dibuat sebuah proyek Android Studio bernama StateHoistingApp menggunakan Jetpack Compose. Percobaan dimulai dari sebuah Composable CounterScreen yang bersifat stateful (variabel state dideklarasikan di dalam Composable itu sendiri), kemudian secara bertahap state dan event tersebut dipindahkan (di-hoist) ke Composable pemanggilnya, yaitu fungsi onCreate() pada MainActivity, sehingga CounterScreen menjadi Composable yang stateless dan dapat digunakan kembali (reusable).

1. Jalankan program, klik tombol "Increment" beberapa kali.

(a) Apa yang terjadi setiap kali tombol "Increment" diklik?

(b) Apa yang menyebabkan hal tersebut terjadi?

(c) Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka?

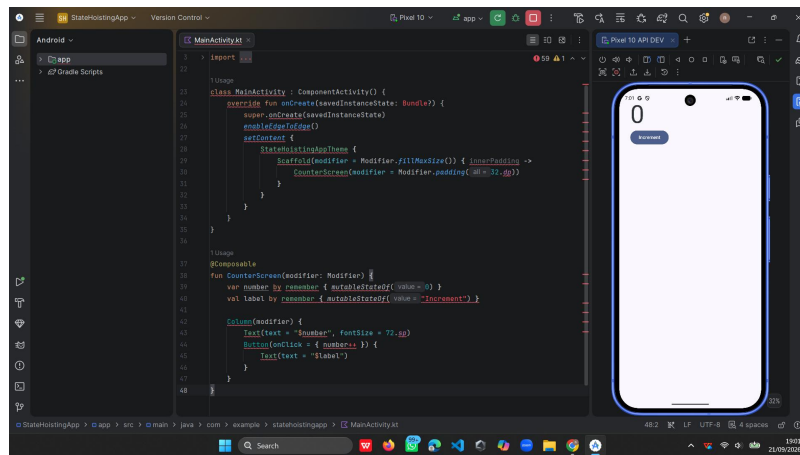
(d) Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button?

Jawaban : Setiap kali tombol "Increment" diklik, angka yang ditampilkan pada layar bertambah satu dari nilai sebelumnya.

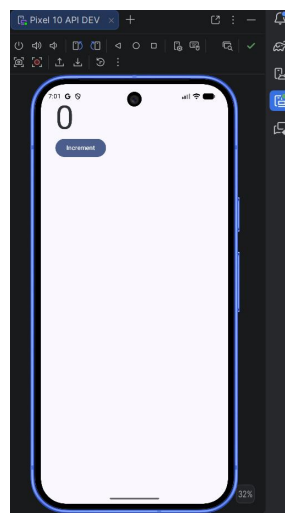
Jawaban : Hal ini terjadi karena tombol tersebut memicu event onClick yang berisi ekspresi `number++`, yang mengubah nilai variabel state `number`. Karena `number` dideklarasikan menggunakan `remember` dan `mutableStateOf`, Compose mendeteksi perubahan tersebut dan melakukan recomposition pada Composable yang membaca nilai `number`, sehingga tampilan angka pada layar ikut diperbarui.

Jawaban : Variabel state yang berperan adalah `number`, yang bertipe `Int` dan dideklarasikan melalui `var number by remember { mutableStateOf(0) }`.

Jawaban : Variabel state yang berperan adalah `label`, yang bertipe `String` dan dideklarasikan melalui `val label by remember { mutableStateOf("Increment") }`.



Gambar 1. Kondisi awal CounterScreen sebelum tombol Increment diklik (angka masih 0)



Gambar 2. Tampilan aplikasi pada percobaan tahap CounterScreen versi awal (stateful)

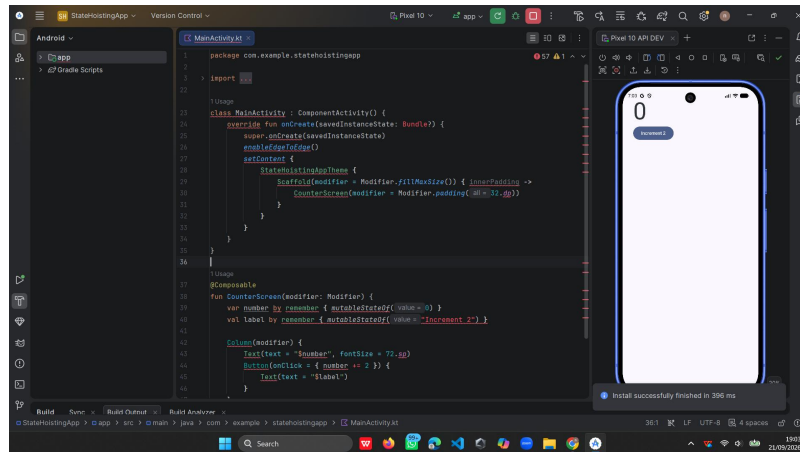
5. Ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai bertambah 2, dan label tombol diganti menjadi "Increment 2".

(a) Perubahan apa yang perlu dilakukan dan di mana?

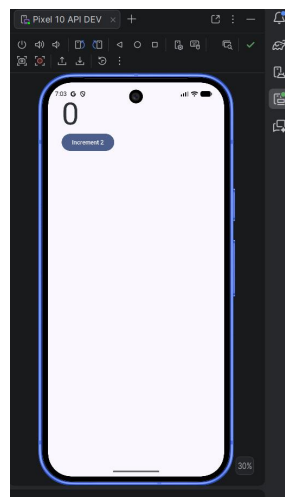
(b) Ambil screenshot sebelum dan sesudah tombol diklik.

Jawaban : Perubahan dilakukan di dalam fungsi Composable CounterScreen itu sendiri, yaitu pada dua bagian: ekspresi di dalam lambda onClick milik Button diubah dari `number++` menjadi `number += 2`, dan nilai awal pada deklarasi `val label by remember { mutableStateOf(...) }` diubah dari "Increment" menjadi "Increment 2". Kedua state, number maupun label, masih tetap dideklarasikan di dalam CounterScreen pada tahap ini, belum dipindahkan ke luar.

Jawaban : Setelah dijalankan, tampilan awal menunjukkan angka 0 dengan label tombol "Increment 2", dan setiap klik menambah angka sebanyak 2 (misalnya dari 0 menjadi 2, lalu 4, dan seterusnya), sesuai dengan perubahan logika yang dilakukan.



Gambar 3. Kode dan tampilan CounterScreen setelah diubah menjadi Increment 2



Gambar 4. Tampilan aplikasi dengan label tombol "Increment 2"

6. Jalankan program setelah dilakukan state dan event hoisting, klik beberapa kali tombol Increment.

- (a) Apakah program menampilkan hal yang sama seperti sebelumnya?
- (b) Di bagian kode manakah proses increment sekarang dilakukan?
- (c) Di bagian kode manakah state Composable Text dan Button sekarang dideklarasikan?

Jawaban : Ya, program tetap berjalan dan menampilkan perilaku yang persis sama seperti sebelum dilakukan hoisting, yaitu angka bertambah setiap kali tombol ditekan dengan

label "Increment". Hasil akhir yang dilihat pengguna tidak berubah sama sekali, walau struktur kode di baliknya berbeda.

Jawaban : Proses increment sekarang dilakukan di dalam fungsi onCreate() pada MainActivity, tepatnya pada lambda onClick = { number++ } yang diteruskan sebagai parameter saat memanggil CounterScreen, bukan lagi di dalam Composable CounterScreen itu sendiri.

Jawaban : Setelah tahap hoisting ini, hanya number yang tetap berstatus sebagai state sungguhan, yaitu dideklarasikan melalui var number by remember { mutableStateOf(0) } di dalam fungsi onCreate(), sebelum pemanggilan Composable CounterScreen. Adapun label sebenarnya sudah tidak lagi berupa state yang dibungkus remember/mutableStateOf pada tahap ini label kini hanya berupa parameter bertipe String biasa yang nilainya ("Increment") dituliskan langsung sebagai literal pada saat memanggil CounterScreen di dalam onCreate(). Composable CounterScreen sendiri hanya menerima number dan label sebagai parameter (number: Int, label: String, onClick: () -> Unit), tanpa mendeklarasikan state apa pun secara mandiri di dalamnya.

7. Diasumsikan ada perubahan kebutuhan fungsional:

- (a) label tombol menjadi "Double",
- (b) nilai awal number dari 0 menjadi 1,
- (c) onClick menggandakan nilai number,
- (d) tuliskan kode yang mendeklarasikan state dan pemanggilan CounterScreen setelah perubahan ini.

Jawaban : Ketiga perubahan tersebut dilakukan seluruhnya di dalam fungsi onCreate(), tanpa menyentuh fungsi Composable CounterScreen sama sekali. Kode setelah perubahan adalah sebagai berikut:

```
var number by remember { mutableStateOf(1) }
```

```
CounterScreen(  
    modifier = Modifier.padding(32.dp),  
    number = number,  
    label = "Double",
```

```
onButtonClick = { number *= 2 }
)
```

8. Jalankan program.

(a) Perlihatkan screenshot sebelum dan sesudah nilai digandakan.

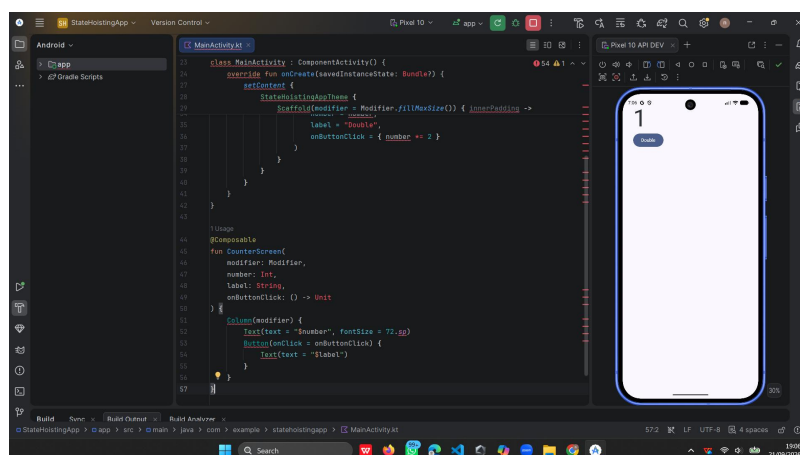
(b) Di bagian manakah perubahan tahap 7 dan tahap 8 dilakukan?

(c) Apakah ada perubahan yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri antara tahap 7 dan tahap 8?

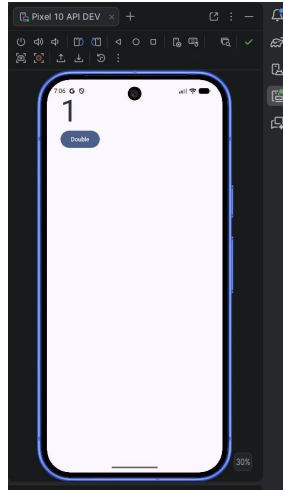
Jawaban : Sebelum tombol "Double" ditekan, layar menampilkan angka 1 sesuai nilai awal yang baru. Setelah tombol ditekan beberapa kali, angka bertambah secara berlipat ganda (1 menjadi 2, lalu 4, 8, 16, dan seterusnya).

Jawaban : Seluruh perubahan pada tahap 7 (hoisting) maupun tahap 8 (perubahan requirement label, nilai awal, dan logika penggandaan) dilakukan di dalam fungsi onCreate() pada MainActivity, yaitu pada bagian deklarasi state number dan pada parameter yang diteruskan ke pemanggilan CounterScreen.

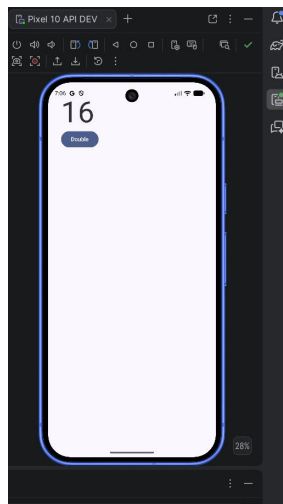
Jawaban : Tidak ada perubahan sama sekali yang dilakukan di dalam fungsi Composable CounterScreen antara tahap 7 dan tahap 8. Fungsi tersebut tetap persis sama, baik signature maupun isinya.



Gambar 5. Kode CounterScreen dengan label "Double" dan tampilan awal angka 1



Gambar 6. Tampilan aplikasi sebelum tombol Double ditekan (angka 1)



Gambar 7. Tampilan aplikasi setelah tombol Double ditekan beberapa kali (angka 16)

10. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat state hoisting dalam perubahan kebutuhan fungsional aplikasi?

Jawaban : Hal ini menunjukkan manfaat utama dari state hoisting, yaitu Composable yang sudah di-hoist menjadi lebih tahan terhadap perubahan kebutuhan (requirement) di masa depan. Karena logika dan data yang bisa berubah-ubah (label, nilai awal, aturan perhitungan) dipindahkan ke Composable pemanggil, maka ketika ada perubahan kebutuhan fungsional, pengembang cukup mengubah bagian pemanggil tanpa perlu menyentuh atau menguji ulang Composable anak yang sudah stabil. Ini membuat Composable seperti CounterScreen menjadi lebih reusable dan lebih mudah dipelihara, karena tanggung jawabnya murni pada tampilan (presentasi), sementara logika bisnis dan sumber data berada di luar dirinya.

B. Percobaan 2 Perubahan Konfigurasi

Percobaan ini melanjutkan kode program dari Percobaan 1 setelah tahap hoisting (tahap nomor 6), dengan fungsionalitas menggandakan nilai number melalui tombol "Double". Tujuannya adalah mengamati apa yang terjadi pada nilai state ketika terjadi perubahan konfigurasi (configuration change) berupa rotasi layar dari posisi portrait ke landscape.

1. Jalankan program dengan posisi portrait, klik beberapa kali tombol "Double" sehingga angka bukan lagi 1. Catat angka tersebut, ambil screenshot.

Jawaban : Setelah tombol "Double" ditekan beberapa kali pada posisi portrait, angka yang ditampilkan berubah dari nilai awal 1 menjadi kelipatan dua dari nilai sebelumnya, misalnya 8, sebagaimana ditunjukkan pada Gambar 9.

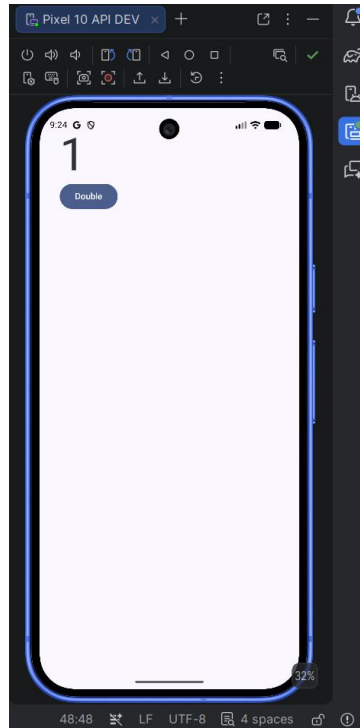
2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari portrait ke landscape.

(a) Catat angka yang ditampilkan, ambil screenshot tampilan landscape-nya.

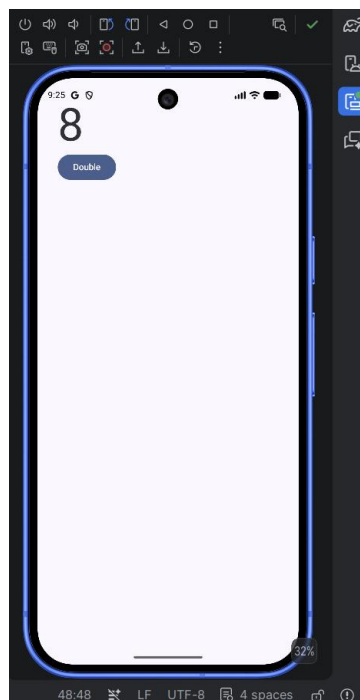
(b) Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawaban : Setelah dirotasi ke posisi landscape, angka yang ditampilkan kembali ke nilai awal, yaitu 1, sebagaimana ditunjukkan pada Gambar 10.

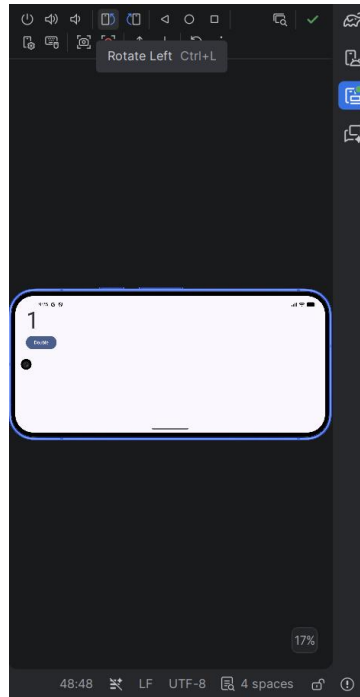
Jawaban : Angka kembali menjadi 1 karena state number pada tahap ini masih dideklarasikan menggunakan remember biasa. Rotasi layar merupakan salah satu bentuk configuration change yang menyebabkan Android menghancurkan (destroy) dan membuat ulang (recreate) Activity yang sedang berjalan. remember hanya mampu mempertahankan nilai suatu state selama composition yang sama masih berlangsung dalam satu siklus hidup Activity, dan tidak memiliki mekanisme untuk menyelamatkan nilainya ketika Activity itu sendiri dihancurkan dan dibuat ulang, sehingga ketika CounterScreen dikomposisi ulang pada Activity yang baru, number kembali dibaca dari nilai awal yang dideklarasikan, yaitu 1.



Gambar 8. Kondisi awal aplikasi (angka 1) sebelum tombol Double ditekan, menggunakan remember biasa



Gambar 9. Kondisi setelah tombol Double ditekan beberapa kali (angka 8), posisi masih portrait, menggunakan remember biasa



Gambar 10. Kondisi setelah dirotasi ke landscape angka kembali menjadi 1 (reset), membuktikan remember tidak bertahan melewati configuration change

Setelah diamati bahwa remember tidak mampu mempertahankan nilai state melewati rotasi layar, kode program diubah dengan mengganti remember menjadi rememberSaveable pada deklarasi state number, agar nilai tersebut dapat bertahan melewati configuration change berikutnya.

3. Setelah kode diubah menggunakan rememberSaveable, lakukan kembali rotasi layar dari portrait ke landscape.

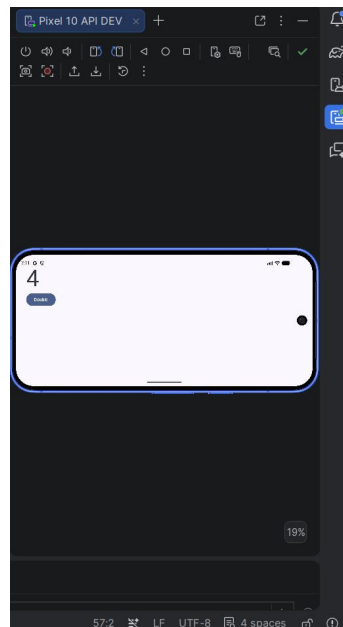
- (a) Catat angka dan screenshot tampilan landscape-nya.
- (b) Berapa angka yang ditampilkan? Mengapa terjadi demikian?
- (c) Apa fungsi rememberSaveable yang membedakannya dari remember?

Jawaban : Setelah rotasi, angka yang ditampilkan tetap sama seperti sebelum dirotasi, tidak kembali ke nilai awal.

Jawaban : Hal ini terjadi karena rememberSaveable, berbeda dari remember biasa, menyimpan nilai state ke dalam Bundle milik Activity (mekanisme yang sama dengan onSaveInstanceState pada Android klasik). Ketika Activity dihancurkan dan dibuat ulang akibat configuration change seperti rotasi layar, Android secara otomatis memulihkan

nilai dari Bundle tersebut, sehingga nilai number yang ditampilkan tetap konsisten dengan kondisi sebelum rotasi terjadi.

Jawaban : Fungsi `rememberSaveable` adalah menyimpan nilai state agar tetap bertahan (survive) melewati proses configuration change maupun proses process death yang dilakukan sistem Android, sesuatu yang tidak dapat dilakukan oleh `remember` biasa. `remember` hanya mempertahankan nilai selama composition yang sama masih berlangsung dalam satu siklus hidup Activity, sedangkan `rememberSaveable` mempertahankan nilai bahkan ketika Activity tersebut dihancurkan dan dibuat ulang, selama nilainya termasuk tipe data yang dapat disimpan ke dalam Bundle (seperti tipe primitif, String, atau tipe yang mengimplementasikan `Parcelable`).



Gambar 11. Tampilan aplikasi pada posisi landscape setelah rotasi, dengan nilai number tetap terjaga berkat `rememberSaveable`

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` tepat digunakan, serta sebutkan manfaatnya!

Jawaban : Salah satu contoh kasus nyata adalah form input pada aplikasi, misalnya form pendaftaran akun yang berisi banyak kolom seperti nama, email, dan nomor telepon. Apabila pengguna sedang mengisi form tersebut lalu secara tidak sengaja memutar layar perangkatnya, penggunaan `rememberSaveable` pada setiap state kolom input akan memastikan data yang sudah diketik pengguna tidak hilang begitu saja akibat Activity dibuat ulang. Manfaat yang diperoleh adalah pengalaman pengguna yang lebih baik, karena pengguna tidak perlu mengulang seluruh proses pengisian data hanya karena

terjadi perubahan konfigurasi layar yang sebenarnya di luar kendali mereka, sehingga aplikasi terasa lebih andal dan tidak kehilangan progres kerja pengguna secara tiba-tiba.

C. Percobaan 3 LaunchedEffect

Percobaan ini melanjutkan kode program dari Percobaan 2 (setelah penerapan `rememberSaveable`), dengan menambahkan fitur side-effect menggunakan `LaunchedEffect`. Percobaan dilakukan dalam dua tahap: pertama, menampilkan pesan `Snackbar` setiap kali nilai number berubah; kedua, mengambil data dari sumber eksternal (disimulasikan melalui kelas `Repo`) satu kali saja saat aplikasi pertama kali dijalankan.

Tahap 1 `LaunchedEffect(number)` dan `Snackbar`

1. Jalankan program.

- (a) Pada saat pertama kali dijalankan, tanpa mengklik tombol "Double", apa yang ditampilkan di layar?
- (b) Mengapa pesan tersebut muncul saat aplikasi pertama kali dijalankan tanpa interaksi apa pun?
- (c) Apa yang memicu pesan `Snackbar` tersebut, dan di bagian kode manakah?

Jawaban : Pada saat aplikasi pertama kali dijalankan, tanpa menekan tombol apa pun, sebuah pesan `Snackbar` langsung muncul di bagian bawah layar bertuliskan "Number 1 is shown!", sesuai dengan nilai awal number.

Jawaban : Pesan tersebut muncul karena `LaunchedEffect(number)` dijalankan setiap kali `composable` pertama kali masuk ke dalam komposisi (initial composition), yang dianggap sebagai bentuk "perubahan" pertama pada key number, sehingga blok kode di dalamnya langsung dieksekusi tanpa memerlukan interaksi pengguna sama sekali.

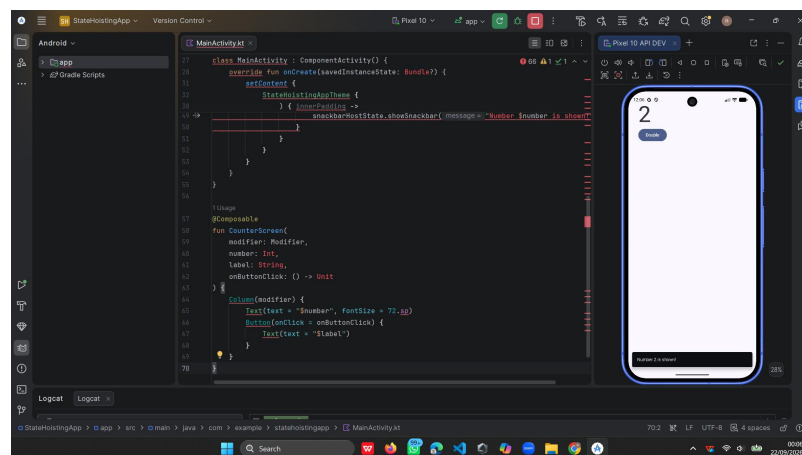
Jawaban : Pesan tersebut dipicu oleh baris kode `LaunchedEffect(number)` { `snackbarHostState.showSnackbar("Number $number is shown!")` } yang dideklarasikan di dalam fungsi `onCreate()`, tepat setelah pemanggilan `Composable CounterScreen`.

2. Klik tombol "Double" satu kali.

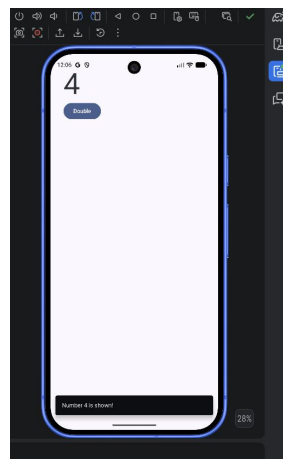
- (a) Apa yang terjadi?
- (b) Mengapa hal tersebut terjadi?

Jawaban : Setelah tombol "Double" ditekan satu kali, nilai number bertambah (digandakan), dan pesan Snackbar baru kembali muncul di bagian bawah layar dengan angka yang sudah diperbarui.

Jawaban : Hal ini terjadi karena `LaunchedEffect(number)` menggunakan number sebagai key. Setiap kali nilai key tersebut berubah (dalam hal ini akibat `number *= 2` pada `onButtonClick`), Compose akan membatalkan coroutine `LaunchedEffect` yang sedang berjalan dan menjalankan ulang blok kode di dalamnya dengan nilai number yang baru, sehingga Snackbar ditampilkan lagi dengan pesan yang sudah diperbarui.



Gambar 12. Tampilan CounterScreen dengan Snackbar sudah diterapkan



Gambar 13. Snackbar "Number 4 is shown!" muncul setelah tombol Double ditekan

Tahap 2 `LaunchedEffect(Unit)` dan `Repo.getData()`

1. Jalankan program.

(a) Pada saat pertama kali dijalankan, tanpa mengklik tombol, apa yang ditampilkan di layar?

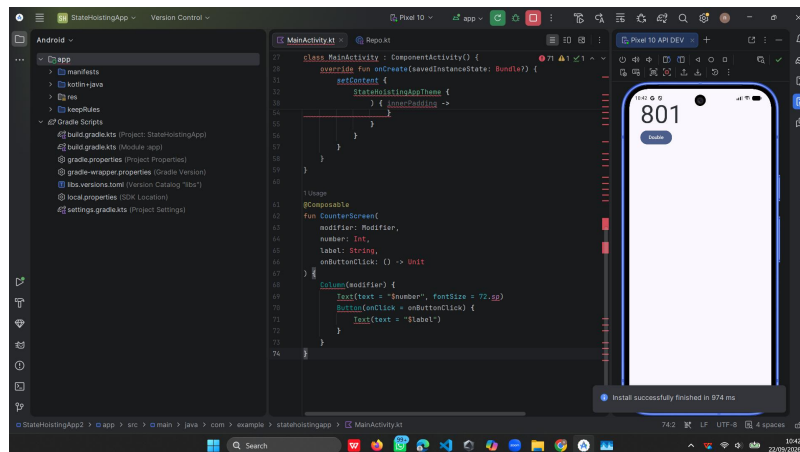
(b) Tunggu dua detik, apa yang tampil setelah itu?

(c) Apa yang memicu perubahan angka dan pesan Snackbar tersebut, dan di bagian kode manakah?

Jawaban : Pada saat aplikasi pertama kali dijalankan, layar menampilkan angka 1 (nilai awal number) beserta Snackbar bertuliskan "Number 1 is shown!", sama seperti pada Tahap 1.

Jawaban : Setelah menunggu sekitar dua detik, angka pada layar berubah secara otomatis menjadi sebuah angka acak antara 100 hingga 999 (pada percobaan ini diperoleh angka 801), tanpa ada interaksi apa pun dari pengguna, dan Snackbar baru kembali muncul menampilkan angka hasil tersebut.

Jawaban : Perubahan ini dipicu oleh blok kode `LaunchedEffect(Unit) { number = Repo.getData() }` yang dideklarasikan setelah `LaunchedEffect(number)` pada tahap sebelumnya. `LaunchedEffect(Unit)` dijalankan satu kali saat composable pertama kali masuk komposisi, memanggil fungsi `suspend Repo.getData()` yang menunda eksekusi selama 2000 milidetik (mensimulasikan proses pengambilan data melalui jaringan) sebelum mengembalikan sebuah angka acak, yang kemudian ditetapkan sebagai nilai baru bagi number. Perubahan nilai number ini pada gilirannya juga memicu `LaunchedEffect(number)` untuk berjalan ulang, sehingga Snackbar baru pun ditampilkan.



Gambar 14. Angka 801 hasil pemanggilan `Repo.getData()` setelah menunggu simulasi delay jaringan selama dua detik

2. Dari kedua percobaan terkait side-effect dengan `LaunchedEffect`:

- (a) Jelaskan perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`.
- (b) Kapan operasi di dalam masing-masing dijalankan?

Jawaban : `LaunchedEffect(state)`, seperti `LaunchedEffect(number)` pada percobaan ini, akan dijalankan ulang setiap kali nilai key yang diberikan (`number`) berubah, karena Compose membandingkan nilai key pada tiap recomposition dan menganggap perubahan nilai sebagai sinyal untuk membatalkan coroutine lama dan memulai coroutine baru. Sebaliknya, `LaunchedEffect(Unit)` menggunakan `Unit` sebagai key, yang nilainya tidak pernah berubah sepanjang siklus hidup composable tersebut, sehingga blok kode di dalamnya hanya dijalankan tepat satu kali, yaitu saat composable pertama kali masuk ke dalam komposisi.

Jawaban : Operasi di dalam `LaunchedEffect(state)` dijalankan setiap kali composable pertama kali masuk komposisi dan setiap kali nilai state yang menjadi key-nya berubah pada recomposition berikutnya. Operasi di dalam `LaunchedEffect(Unit)` hanya dijalankan satu kali saja, yaitu pada saat composable pertama kali masuk komposisi, dan tidak akan dijalankan ulang selama composable tersebut tetap berada dalam komposisi, tidak peduli berapa kali recomposition terjadi setelahnya.

D. Kesimpulan

Melalui rangkaian tiga percobaan pada modul ini, dapat disimpulkan bahwa state hoisting merupakan praktik penting dalam Jetpack Compose untuk memisahkan tanggung jawab tampilan (presentasi) dari logika dan sumber data, sehingga Composable menjadi lebih reusable dan tahan terhadap perubahan kebutuhan fungsional tanpa perlu diubah strukturnya. Perbedaan antara `remember` dan `rememberSaveable` menunjukkan pentingnya memilih mekanisme penyimpanan state yang tepat, terutama untuk data yang harus tetap bertahan melewati proses configuration change seperti rotasi layar. Adapun `LaunchedEffect` memperkenalkan cara yang aman dan terstruktur untuk menjalankan operasi asynchronous (side-effect) di dalam lingkungan Compose yang bersifat deklaratif, dengan key sebagai penentu kapan operasi tersebut perlu dijalankan ulang `LaunchedEffect(state)` untuk operasi yang perlu bereaksi terhadap perubahan data, dan `LaunchedEffect(Unit)` untuk operasi yang cukup dijalankan satu kali saja, seperti pengambilan data awal dari sumber eksternal.